Khmelnytskyi National University
Department of Information Technonogy

Report

Laboratory work №7

Discipline: “Programming”

Topic: “*Strings*”

Completed: 1st year student, group SE-26-1 Nykon Hrytsyshyn
Name, Surname

Checked: Valeriy Martynyuk
Name, Surname

Khmelnytskyi, 2026

Purpose: study of basic algorithms for working with strings.

Technical Environment: The project is implemented in accordance with the C11 standard and configured for cross-platform compilation using CMake and the Ninja build system. The application is built and verified using modern C compilers (GCC, Clang, and MSVC) with strict diagnostic flags enabled, ensuring high code quality, portability, and compatibility across operating systems.

Note: The software implementation is designed using a modular architecture with robust input validation and an interactive task execution engine. Detailed flowcharts illustrating the overall execution logic, along with full source code for all program components (tasks.h, utils.h, main.c, runner.c and utils.c), are provided in Appendices A…F.

In Table 1, general information about the tasks is provided.

Table 1 – Tasks overview

Task	What it does
1	Replaces the symbols `+`, `-`, `*` of the line with spaces
2	Splits a text into words separated by `:`, `;`, `-` and a space; print and count them
3	Finds the shortest and the longest word of the line, print them and their lengths

The following sections provide a detailed breakdown of each implemented task. For every task, the problem statement is followed by the algorithm's block diagram, the corresponding C source code, and demonstration screenshots of the program execution results.

Task 1

Delete the symbols "+", "-", "*" from the given line, and place to put a space.

1.1. Block diagram of the program algorithm

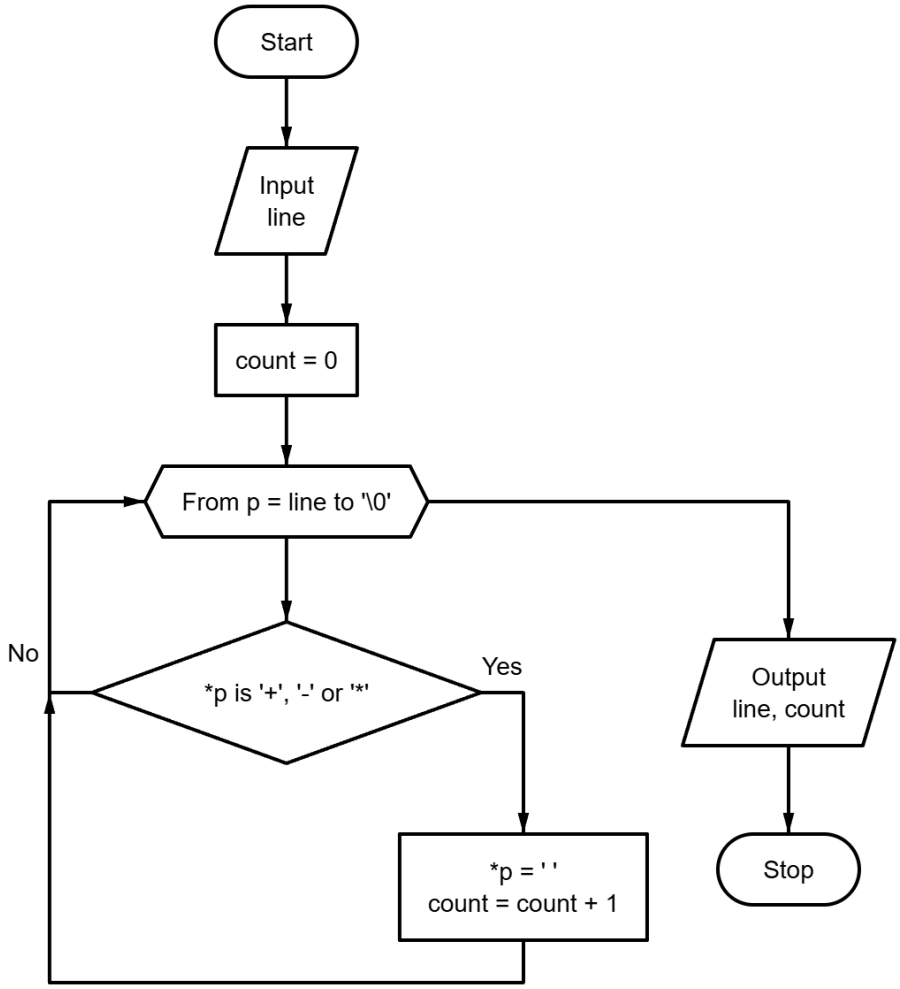


Fig. 1 – Block diagram of the program algorithm

1.2. Program text

```
#define REPLACED_SYMBOLS "+-*"

#include "tasks.h"
#include "utils.h"

#include <stdio.h>
#include <string.h>

static long replace_symbols(char *const text) {
    long count = 0;

    for (char *p = text; *p != '\0'; ++p) {
        if (strchr(REPLACED_SYMBOLS, *p) != NULL) {
            *p = ' ';
            ++count;
```

```
        }
    }

    return count;
}

void run_task1(void) {
    char text[MAX_LINE_SIZE];

    puts(
        "\n=== Task 1: replace the symbols \"+\", \"-\", \"*\"
with spaces "
        "===\n"
    );

    if (!read_text_line("Enter the line: ", text,
sizeof(text))) {
        return;
    }

    const long count = replace_symbols(text);

    printf("\nResult: \"%s\"\n", text);
    printf("Replaced symbols: %ld\n", count);
}
```

1.3. Program results

```
=== Task 1: replace the symbols "+", "-", "*" with spaces ===

Enter the line: 2+3-4*5 a+b

Result: "2 3 4 5 a b"
Replaced symbols: 4
```

```
=== Task 1: replace the symbols "+", "-", "*" with spaces ===

Enter the line: hello world

Result: "hello world"
Replaced symbols: 0
```

Fig. 2 – Task 1 results

Task 2

Given a text whose words are separated by a colon, a semicolon, a hyphen, and a space. Highlight, print and count the number of words. Display the results on the screen.

2.1. Block diagram of the program algorithm

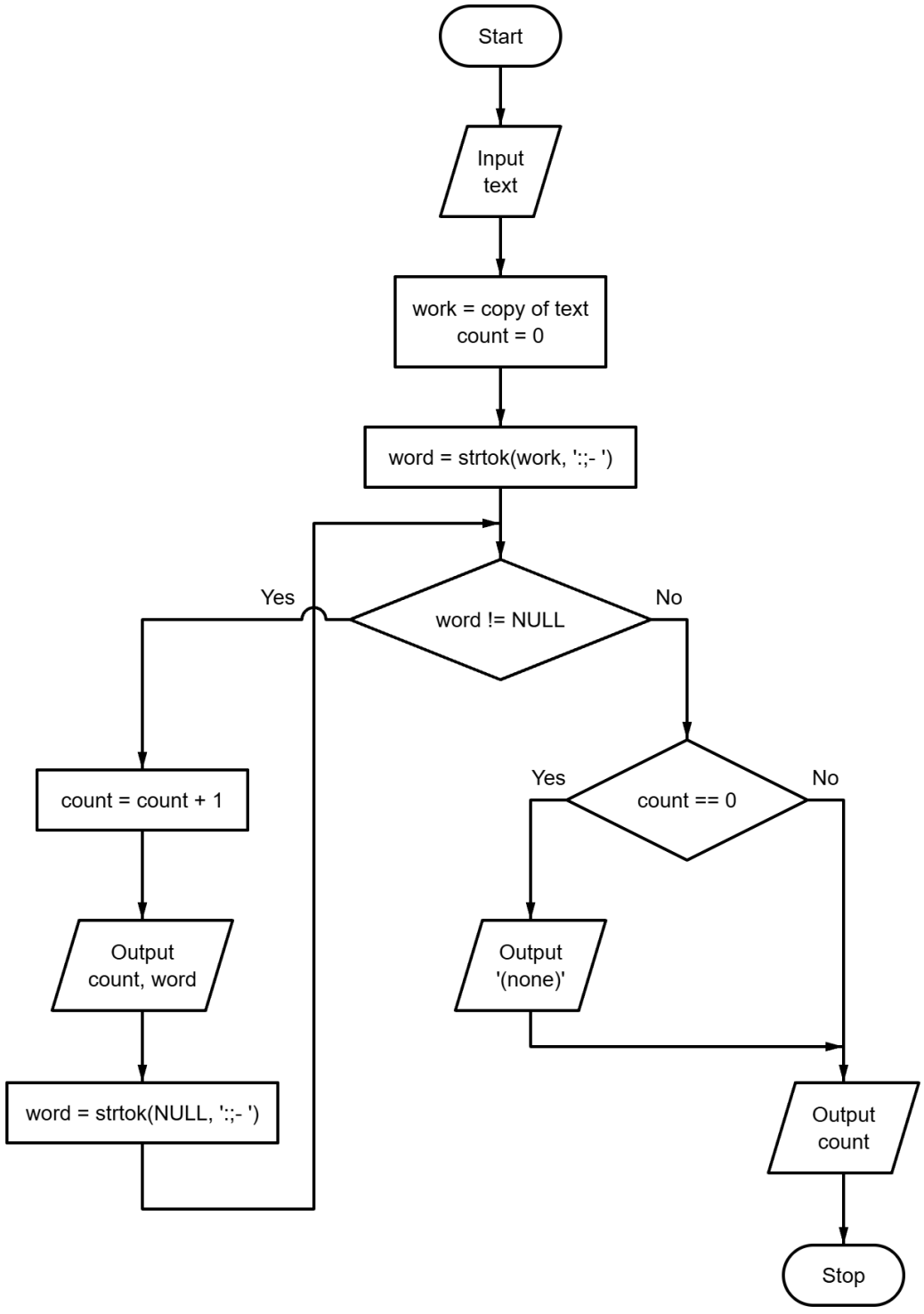


Fig. 3 – Block diagram of the program algorithm

2.2. Program text

```
#define WORD_DELIMITERS ":;- "

#include "tasks.h"
#include "utils.h"

#include <stdio.h>
#include <string.h>

void run_task2(void) {
    char text[MAX_LINE_SIZE];
    char work[MAX_LINE_SIZE];

    puts(
        "\n=== Task 2: words separated by \":\", \";\", \"-\" and
a space ===\n"
    );

    if (!read_text_line("Enter the text: ", text, sizeof(text))) {
        return;
    }

    snprintf(work, sizeof(work), "%s", text);
    puts("\nWords:");

    long count = 0;

    for (const char *word = strtok(work, WORD_DELIMITERS); word !=
NULL;
         word = strtok(NULL, WORD_DELIMITERS)) {
        ++count;
        printf("%ld) %s\n", count, word);
    }

    if (count == 0) {
        puts("(none)");
    }

    printf("\nNumber of words: %ld\n", count);
}
```

2.3. Program results

```
=== Task 2: words separated by ":", ";", "-" and a space ===

Enter the text: one:two;three-four five

Words:
1) one
2) two
3) three
4) four
5) five

Number of words: 5
```

```
=== Task 2: words separated by ":", ";", "-" and a space ===

Enter the text: :;-

Words:
(none)

Number of words: 0
```

Fig. 4 – Task 2 results

Task 3

Find the shortest and longest word in the given line, output these words and their length.

3.1. Block diagram of the program algorithm

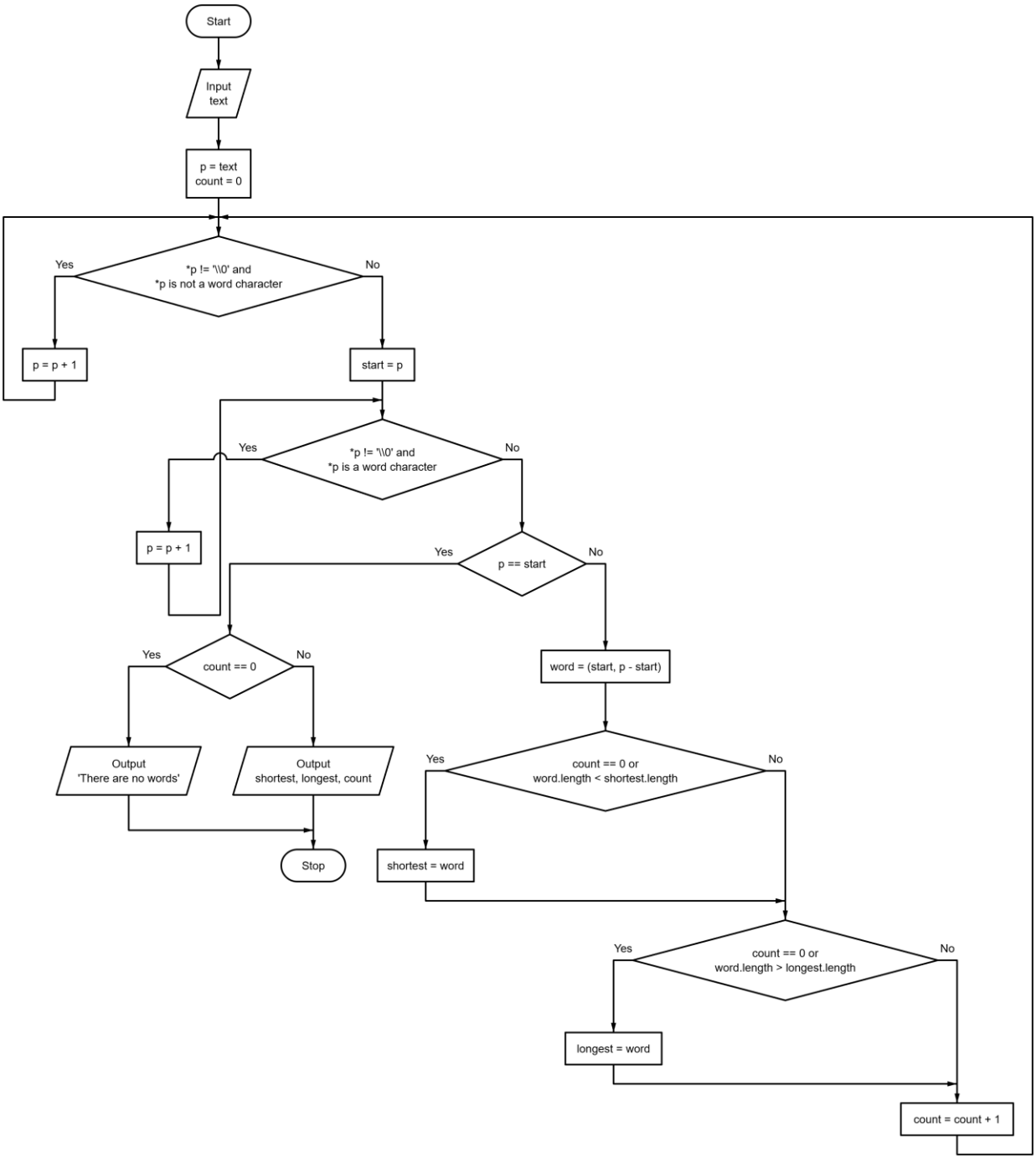


Fig. 5 – Block diagram of the program algorithm

3.2. Program text

```
#define NON_ASCII_BIT 0x80

#include "tasks.h"
```

```
#include "utils.h"

#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>

typedef struct {
    const char *start;
    size_t bytes;
    size_t length;
} Word;

static bool is_word_char(const unsigned char c) {
    return isalnum(c) || c >= NON_ASCII_BIT;
}

static bool next_word(const char **const position, Word *const
out_word) {
    const char *p = *position;

    while (*p != '\0' && !is_word_char((unsigned char)*p)) {
        ++p;
    }

    const char *const start = p;

    while (*p != '\0' && is_word_char((unsigned char)*p)) {
        ++p;
    }

    *position = p;

    if (p == start) {
        return false;
    }

    out_word->start = start;
    out_word->bytes = (size_t)(p - start);
    out_word->length = text_length(start, out_word->bytes);

    return true;
}

static void print_word(const char *const title, const Word word) {
    printf(
        "%s: \"%.*s\", length %zu\n",
        title,
```

```
        (int)word.bytes,
        word.start,
        word.length
    );
}

void run_task3(void) {
    char text[MAX_LINE_SIZE];

    puts("\n=== Task 3: the shortest and the longest word ===\n");

    if (!read_text_line("Enter the text: ", text, sizeof(text))) {
        return;
    }

    const char *position = text;
    Word word;
    Word shortest = {0};
    Word longest = {0};
    long count = 0;

    while (next_word(&position, &word)) {
        if (count == 0 || word.length < shortest.length) {
            shortest = word;
        }

        if (count == 0 || word.length > longest.length) {
            longest = word;
        }

        ++count;
    }

    puts("");

    if (count == 0) {
        puts("There are no words in the text.");
        return;
    }

    print_word("Shortest word", shortest);
    print_word("Longest word", longest);
    printf("Number of words: %ld\n", count);
}
```

3.3. Program results

```
=== Task 3: the shortest and the longest word ===

Enter the text: I love programming in C language

Shortest word: "I", length 1
Longest word: "programming", length 11
Number of words: 6
```

```
=== Task 3: the shortest and the longest word ===

Enter the text: ab, cd! ef; gh.

Shortest word: "ab", length 2
Longest word: "ab", length 2
Number of words: 4
```

Fig. 6 – Task 3 results

Conclusions

During the execution of this laboratory work, practical skills were acquired in handling null-terminated character strings and implementing fundamental string processing algorithms in C11. The tasks demonstrated effective functional decomposition across various string manipulation problems, including in-place character searching and replacement using strchr(), text tokenization with multi-delimiter parsing via strtok(), and custom word boundary processing to evaluate string metrics (identifying minimum and maximum word lengths with multibyte character support).

Furthermore, experience was gained in building cross-platform modular C software structured with CMake and Ninja, enforcing strict compiler diagnostics, and implementing robust input validation within an interactive task execution engine.

In Table 2, all results of the completed tasks are highlighted.

Table 2 – Results overview

Task	Input	Output
1	2+3-4*5 a+b	2 3 4 5 a b, replaced symbols: 5
	hello world	hello world, replaced symbols: 0
2	one:two;three-four five	words one, two, three, four, five; number of words: 5
	:;-	(none), number of words: 0
3	I love programming in C language	shortest I (1), longest programming (11), words: 6
	ab, cd! ef; gh.	shortest ab (2), longest ab (2): the first of equal ones, words: 4

APPENDIX A

Block diagram of the main program control flow algorithm

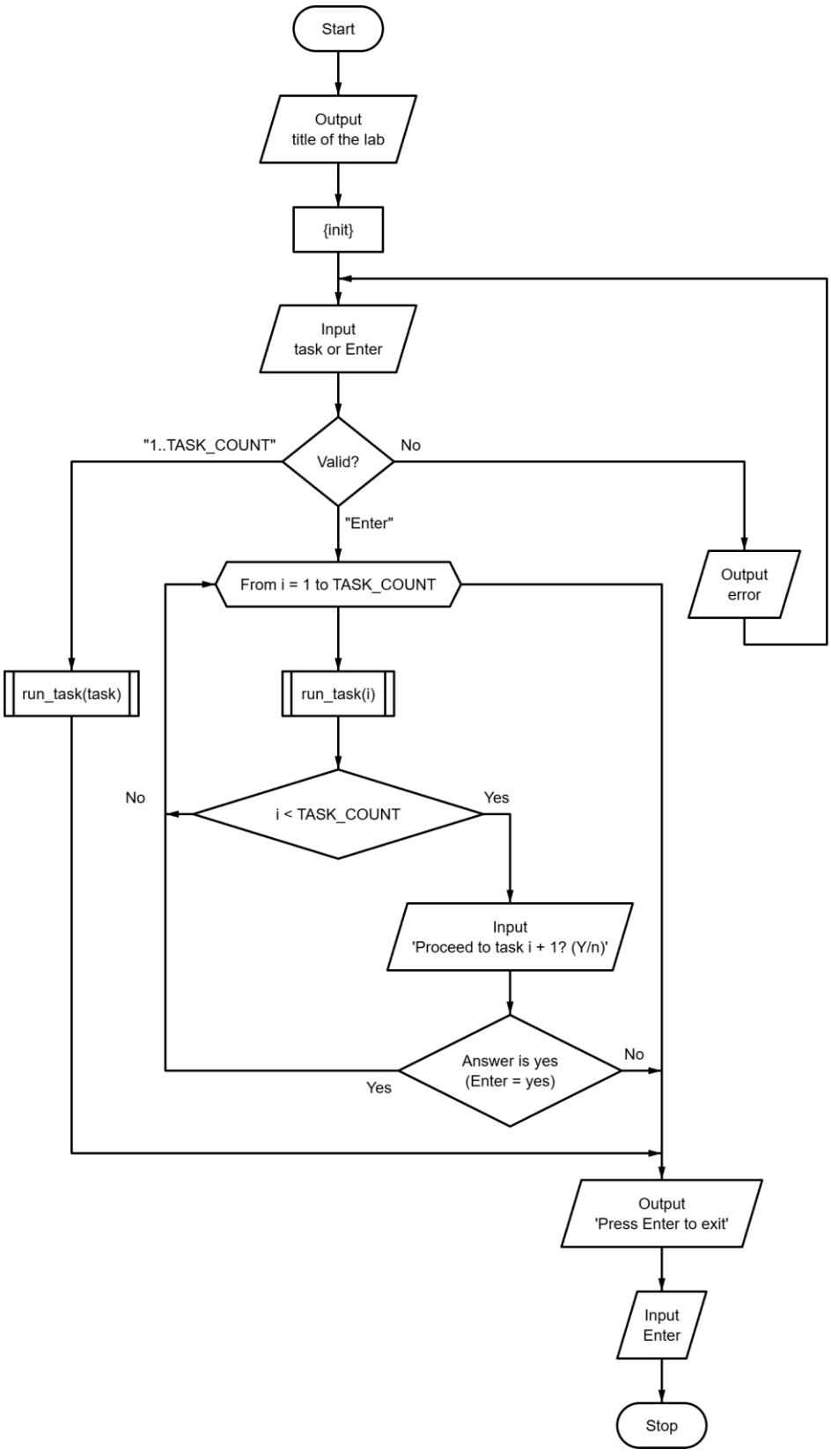


Fig. A.1 – Block diagram of the main program control flow algorithm

APPENDIX B

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX C

Source code of the “utils.h” file

```
#ifndef UTILS_H
#define UTILS_H

#include <stdbool.h>
#include <stddef.h>

#define MAX_LINE_SIZE 256

bool read_double(const char *prompt, double *out_value);
bool read_doubles(const char *prompt, size_t count, ...);
bool read_long(const char *prompt, long *out_value);
bool read_longs(const char *prompt, size_t count, ...);
bool read_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value
);
bool read_optional_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value,
    bool *out_provided
);

bool read_text_line(const char *prompt, char *buffer, size_t
size);
size_t text_length(const char *text, size_t bytes);

bool confirm(const char *prompt, bool default_answer);

#endif // UTILS_H
```

APPENDIX D

Source code of the “main.c” file

```
#include "tasks.h"
#include "utils.h"
#include <locale.h>
#include <stdbool.h>
#include <stdio.h>
#define PROMPT_SIZE 96

int main(void) {
    puts(
        "========================================\n"
        " Laboratory work 7: Strings (variant 9) \n"
        "========================================\n"
    );

    setlocale(LC_CTYPE, "");

    char prompt[PROMPT_SIZE];
    snprintf(
        prompt,
        sizeof(prompt),
        "Enter task number (1-%d) or press Enter to run all tasks
[ALL]: ",
        TASK_COUNT
    );

    long task = 0;
    bool provided = false;

    if (!read_optional_long_in_range(prompt, 1, TASK_COUNT, &task,
&provided)) {
        return 1;
    }

    if (provided) {
        run_task((int)task);
    } else {
        run_all_tasks();
    }

    printf("\nPress Enter to exit...");
    getchar();

    return 0;
}
```

APPENDIX E

Source code of the “runner.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);

        if (i < TASK_COUNT) {
            char prompt[64];
            snprintf(
                prompt, sizeof(prompt), "\nProceed to task %d?
(Y/n): ", i + 1
            );

            if (!confirm(prompt, true)) {
                break;
            }
        }
    }
}
```

APPENDIX F

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { MAX_READ_VALUES = 8 };

typedef bool (*token_parser_fn)(const char *str, char **endptr,
void *out);

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }

        ptr++;
    }

    return true;
}

static bool parse_double_token(const char *str, char **endptr,
void *out) {
    errno = 0;
    *(double *)out = strtod(str, endptr);

    return *endptr != str && errno == 0;
}

static bool parse_long_token(const char *str, char **endptr, void
*out) {
    errno = 0;
    *(long *)out = strtol(str, endptr, 10);

    return *endptr != str && errno == 0;
}

static bool read_line_values(
    const char *prompt,
```

```
    const size_t count,
    void *const outs[],
    const token_parser_fn parse
) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    const char *ptr = buffer;

    for (size_t i = 0; i < count; ++i) {
        if (outs[i] == NULL) {
            return false;
        }

        char *endptr;
        if (!parse(ptr, &endptr, outs[i])) {
            return false;
        }

        ptr = endptr;
    }

    return is_buffer_empty(ptr);
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];
```

```
    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, double *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_double_token);
}

bool read_long(const char *prompt, long *out_value) {
    return read_longs(prompt, 1, out_value);
}

bool read_longs(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, long *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_long_token);
}

bool read_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value
) {
    while (true) {
        if (read_long(prompt, out_value)) {
            if (*out_value >= min && *out_value <= max) {
```

```
                return true;
            }

            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else if (feof(stdin)) {
            return false;
        } else {
            puts("Invalid input. Please enter an integer.");
        }
    }
}

bool read_optional_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value,
    bool *out_provided
) {
    char buffer[256];

    while (true) {
        printf("%s", prompt);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        if (is_buffer_empty(buffer)) {
            *out_provided = false;
            return true;
        }

        char *endptr;
        long value;

        if (!parse_long_token(buffer, &endptr, &value) ||
            !is_buffer_empty(endptr)) {
            puts("Invalid input. Please enter an integer or press
Enter.");
        } else if (value < min || value > max) {
            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else {
            *out_value = value;
            *out_provided = true;
```

```
            return true;
        }
    }
}

bool read_text_line(
    const char *const prompt,
    char *const buffer,
    const size_t size
) {
    while (true) {
        printf("%s", prompt);

        if (fgets(buffer, (int)size, stdin) == NULL) {
            return false;
        }

        char *const newline = strchr(buffer, '\n');

        if (newline != NULL) {
            *newline = '\0';
            return true;
        }

        if (feof(stdin)) {
            return true;
        }

        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF)
            ;

        printf("Error: the line must be shorter than %zu
characters.\n", size);
    }
}

size_t text_length(const char *text, size_t bytes) {
    size_t length = 0;

    mblen(NULL, 0);

    while (bytes > 0) {
        int step = mblen(text, bytes);

        if (step <= 0) {
            step = 1;
```

```
        }

        text += step;
        bytes -= (size_t)step;
        ++length;
    }

    return length;
}

static bool equals_ignore_case(
    const char *str,
    const size_t len,
    const char *word
) {
    if (strlen(word) != len) {
        return false;
    }

    for (size_t i = 0; i < len; ++i) {
        if (tolower((unsigned char)str[i]) != word[i]) {
            return false;
        }
    }

    return true;
}

bool confirm(const char *prompt, const bool default_answer) {
    char buffer[16];

    while (true) {
        if (prompt != NULL) {
            printf("%s", prompt);
        }

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        const bool overflow = strchr(buffer, '\n') == NULL &&
!feof(stdin);
        if (overflow) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF)
                ;
        } else {
```

```
            const char *start = buffer;
            while (isspace((unsigned char)*start)) {
                start++;
            }

            size_t len = strlen(start);
            while (len > 0 && isspace((unsigned char)start[len -
1])) {
                len--;
            }

            if (len == 0) {
                return default_answer;
            }

            if (equals_ignore_case(start, len, "y") ||
                equals_ignore_case(start, len, "yes")) {
                return true;
            }

            if (equals_ignore_case(start, len, "n") ||
                equals_ignore_case(start, len, "no")) {
                return false;
            }
        }

        printf("Please answer 'y' or 'n'.\n");
    }
}
```